Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Strategy (davranışsal kalıp)
  - Amaç: aynı işi yapan birden fazla algoritma varsa, bu algoritmaları onları kullanan istemcilerden soyutlamak ve kullanılan algoritmayı çalışma anında değiştirebilmek.
  - Hocanın gösterdiği örnek: metin düzenleyicilerdeki alt satıra geçme (satır bölme) algoritmaları. Word boşlukları genişletip daraltarak satırı iki yana yaslar; WordPad'de "cetvele kaydır", "pencereye kaydır" ve "kaydırma yok" seçenekleri vardır; kelime heceden de bölünebilir.
  - Örnek çözüm (ekrandan, 00:13:15): `Uygulama` sınıfında `- bölmeleyici : SatırBölümleme` alanı ve `+ metinDüzenle()` metodu vardır; `metinDüzenle()` içinde `bölmeleyici.bölmele()` çağrılır. `«abstract» SatırBölümleme` sınıfında `+ bölmele() {abstract}` vardır; alt sınıfları `KesmeliBöl`, `KelimedenBöl`, `SınırdanBöl`'dür.
  - Varsayılan bir bölme algoritması varsa üst sınıfın ve metodun soyut olması zorunlu değildir, ama kalıbın genel yapısı soyut üst sınıftır (ekrandan, 00:13:15).
  - Kalıp yapısı (ekrandan, 00:16:10): `Context` (`ContextInterface()`), `strategy` ilişkisiyle soyut `Strategy` (`AlgorithmInterface()`), alt sınıfları `ConcreteStrategyA/B/C`. `Strategy` tüm algoritmaların ortak arayüzünü belirler; `ContextInterface()` gerekli hallerde algoritma gerçeklemesine durum bilgisini açan metotlardır.
  - Gerçekleme ayrıntıları (ekrandan, 00:20:00): `Strategy` ve `Context` seçilen algoritmayı gerçeklemek için etkileşir, yani ilişki çoğu zaman çift yönlüdür. `Context`'i kullanan istekçi nesneler ona bir `ConcreteStrategy` nesnesi verir.
  - Uygulanabileceği anlar: bir algoritmanın değişik çeşitlemelerine ihtiyaç varsa; istekçilerin algoritmanın gerçeklenmesini bilmemesi gerekiyorsa.
  - Zayıf yönleri: istekçiler stratejilerin varlığından haberdar olmak zorundadır; `Context`'in açtığı durum bilgisinin tümüne her strateji ihtiyaç duymayabilir, push model kullanılıyorsa haberleşmede ek yük oluşur.
- Builder ile Strategy karşılaştırması
  - İki kalıbın sınıf şemaları neredeyse aynıdır: `RTFReader` yerine `Uygulama`, `TextConverter` yerine `SatırBölümleme`, somut dönüştürücüler yerine somut bölücüler gelir.
  - Fark: Builder'da yönetici (`RTFReader`) karmaşık ürünün her parçasının oluşturulmasını adım adım denetler. Strategy'de istemci bütün işi strateji gerçeklemesine verir, denetim akışının çoğunu strateji yürütür ve gerekli bilgiyi context'ten sorgular.
  - Benzer şemalı kalıpları birbirinden ayıran şey amaçları, kullanım yerleri ve gerçekleme ayrıntılarıdır.
- Strategy örneği: kişisel finans programı (ekrandan, 00:30:00; kaynak DP Explained)
  - Uluslararası pazara hitap eden bir kişisel finans programı farklı ülkelerin vergi kurallarıyla çalışmalıdır. Ana program `TaskController`'dır.
  - Çözüm 1, switch-case: `enum Country { TR, US, EU }`, `TaskController` içinde `code` alanı, `setCountry()` ve `taxAmount(double income)`. Oranlar: TR için `income * 0.18`, EU için `0.15`, US için `0.12`, varsayılan `0.10`.
  - Switch-case'in sorunu: gerçekte her durum karmaşık olacağından metot devasa olur ve her yeni ülke yeni bir enum değeri ile yeni bir durum ister.
  - Çözüm 2, Strategy (ekrandan, 00:33:30): `interface TaxRule { double taxAmount(double income); }`; `TaskController` `private TaxRule tax` alanını tutar, `setTax()` ile alır ve `taxAmount()` işi `tax.taxAmount(income)` ile stratejiye iletir; `TRrules implements TaxRule` `income * 0.18` döndürür.

  - Yatırım, döviz, emeklilik gibi başka mali işlemler varsa her işlem için Strategy kalıbı ayrıca kurulur (`TRInvestment`, `USInvestment` gibi). Tek bir arayüzde bütün işlemleri toplamak da mümkündür ama "böl ve yönet" gereği ayrı kalıplar önerilir; bu zorunlu değildir.
- Singleton ile Strategy birlikte (ekrandan, 00:40:00)
  - Bir ülkenin vergi kanunu tektir; bu yüzden strateji gerçeklemeleri Singleton olmaya adaydır.
  - `TRrules`: `private static TRrules instance`, `private` kurucu, `public static TRrules getInstance()` (`instance == null` ise oluşturur) ve `taxAmount()`. `EUrules` ve `USrules` de benzer biçimde gerçeklenir.
- State (davranışsal kalıp)
  - Amaç: bir nesnenin iç durumu değiştiğinde davranışının da değişmesini sağlamak. Burada küçük parametrik değişiklikler değil, nesnenin çalışma biçiminin tamamen değişmesi kastedilir.
  - Örnek: TCP bağlantısı `Established`, `Listen` ve `Closed` durumlarından birinde olur ve her durumda farklı çalışır. Switch-case yerine "değişeni bul ve sarmala" ilkesiyle soyut bir `TCPState` sınıfı yazılır; bağlantı `open`, `close`, `acknowledge` gibi işleri o anki durum nesnesine iletir.
  - Durumlar arasında geçiş kuralları vardır. State'i Strategy'den ayıran en önemli nokta budur.
  - Kalıp yapısı (ekrandan, 00:50:00): `Context` (`Request()` → `state->Handle()`), `state` ilişkisiyle soyut `State` (`Handle()`), alt sınıfları `ConcreteStateA/B`. Context, davranışı durum bilgisiyle değişen varlıktır. Durum değişiminden genellikle Context sorumludur; aksi halde durumlar geçişi kendileri yapar.
  - Kullanım: nesnenin çalışma biçimi çalışma anında durumuna göre değişiyorsa; durumuna bağlı uzun ve çok seçenekli karar komutları gerekiyorsa.
- State ile Strategy farkları
  - State'te context durum değiştikçe kendi somut durumunu kendisi seçer. Strategy'de hangi stratejinin kullanılacağı dışarıdan verilir.
  - State'te geçişler açıkça kodlanır (context ya da durum sınıflarında); somut durumlar arasında bağımlılık ilişkileri olabilir. Strategy'de geçiş bilgisi daha az açıktır.
  - State'te durum geçişleri sıktır, Strategy'de strateji değişimi seyrektir.
- State örneği: PIM yazılımı (ekrandan, 00:54:00; kaynak Applied Java Patterns)
  - Kişisel bilgi yöneticisindeki kayıtlar `Clean` (temiz) ya da `Dirty` (kirli: değiştirilmiş ama kaydedilmemiş) durumundadır.
  - Sınıf şeması: soyut `State` (`edit()`, `save(f : File, s : Serializable)`), alt sınıfları `CleanState` ve `DirtyState`; `PIMApplication` (`+ CLEAN : State`, `+ DIRTY : State`, `- currentState : State`, `getCurrentState()`, `setCurrentState(state : State)`).
  - `CleanState.edit()` durumu `DIRTY` yapar ve `super.edit()` çağırır; `save()` `super.save(f, s)` sonrasında durumu `CLEAN` yapar (ekrandan, 01:00:00). Slayttaki ilk hâlde `DirtyState` boş bir işaretleyici sınıftı (`//marker class`).
  - Hoca derste bir hata buldu: `DirtyState` boş kalınca kalıtımla gelen `save()` durumu `CLEAN`'e çekmiyordu. Düzeltmede `DirtyState`'e `save()` eklendi: `super.save(f, s)` ve ardından `setCurrentState(PIMApplication.CLEAN)` (ekrandan, 01:10:00). Kodda geçen `StateFactory` şemada yoktu; hoca bunun `PIMApplication` olması gerektiğini söyledi.
  - İki `save()` aynı olduğu hâlde üst sınıfa taşınmadı: üst sınıf alt sınıfları bilmemelidir ve durum geçişleri ya durumlarda ya da context'te tutulmalıdır.
- Template Method (davranışsal kalıp)
  - Amaç: bir algoritmanın ya da iş akışının bazı adımlarını belirleyip geri kalan adımların ayrıntılarını alt sınıflara bırakmak.
  - GoF örneği: farklı belge türleriyle çalışan uygulamalar için çerçeve program. `Application` ve `Document` soyuttur. `OpenDocument()` şablon metodu sırayla `CanOpenDocument()` denetimini, belgenin oluşturulmasını, listeye eklenmesini, açılış öncesi hazırlıkları ve belgenin açılıp okunmasını yürütür; denetim, oluşturma ve hazırlık adımları belgeye bağlı olduğu için soyuttur.
  - Kalıp yapısı (ekrandan, 01:36:00): `AbstractClass` (`TemplateMethod()`, `PrimitiveOperation1()`, `PrimitiveOperation2()`) ve `ConcreteClass`. Şablon metot algoritmanın değişmeyen kısmını gerçekler, alt düzey ayrıntıları alt sınıflara aktarır.
  - Gerçekleme: alt düzey metotlar Java'da `abstract`, C++'ta "pure virtual" olmalıdır.

Şablon metot alt sınıflarca yeniden tanımlanamamalıdır, Java'da `final` olmalıdır. Sınıfın kendisi `final` olmamalıdır.
  - Kullanım: algoritmanın değişmez kısımları bir kez kodlanıp değişen kısımlar alt sınıflara açık bırakılacaksa; yeniden yapılandırmada kardeş sınıflarda tutarsız yinelenen kod görüldüğünde ortak kısım üst sınıfta toplanacaksa.
- Template Method örneği: veritabanı sorgusu (ekrandan, 01:40:00; kaynak DP Explained)
  - Program farklı VTYS'lerle çalışmalıdır; bağlantı ve SQL komutlarını biçimlendirme, özellikle saklı yordam (SP) dilleri farklıdır.
  - Değişmeyen adımlar: bağlantı komutu biçimlendirilir (CONNECT), VT'ye gönderilir, seçme komutu biçimlendirilir (SELECT), VT'ye gönderilir, seçilen veri kümesi döner.
  - Ortak temel adımların olması Strategy yerine Template Method'u daha doğru seçim kılar.
  - Çözüm (ekrandan, 01:45:15): `abstract class QueryTemplate` içinde `public final void doQuery(String DBname, String querySpec)` ve paket görünürlüğünde `abstract String formatConnect(String DBname)`, `abstract String formatSelect(String querySpec)`. Somut sınıflar Oracle, SQL Server gibi VTYS'ler için yazılır; ana uygulama `QueryTemplate` tipinde bir alan tutar.
- Visitor (davranışsal kalıp)
  - Amaç: zincir ya da ağaç biçimindeki bir nesne yapısının elemanları üzerinde, elemanların sınıflarını değiştirmeden yeni işlemler tanımlamak. Gerekçe: kaynak kodu elde olmayabilir, geriye uyumluluk korunmak istenebilir ya da altyapı bir kez kurulup ileride yeni istekler kolayca eklenmek istenebilir.
  - GoF örneği: derleyici. Kaynak kod, düğümleri farklı token türleri olan bir ayrıştırma ağacına dönüşür. Örneğin `x = 3 + 5` için atama düğümü, hedef `x` ve iki ucunda 3 ile 5 olan toplama düğümü oluşur. Ağaç tip denetimi, kod üretimi ve renkli gösterim (pretty printing) için birkaç kez dolaşılır. `+` iki yanında string varsa birleştirme anlamına gelir.
  - İşlemleri düğüm sınıflarına dağıtmak bakım kabusudur; bazı düğümlerde bazı işlemler gerekmediği için boş gövdeli metotlar da oluşur.
  - Örnek çözüm (ekrandan, 02:00:00): soyut `NodeVisitor` (`VisitAssignment(AssignmentNode)`, `VisitVariableRef(VariableRefNode)`), alt sınıfları `TypeCheckingVisitor` ve `CodeGeneratingVisitor`. `Program 1→* Node (Accept(NodeVisitor))`; `AssignmentNode.Accept(v) → v->VisitAssignment(this)`, `VariableRefNode.Accept(v) → v->VisitVariableRef(this)`.
  - Zayıf yön: düğümler ziyaretçinin hangi metodunun kendileri için olduğunu bilmelidir. Bu bağımlılık mümkünse soyut ziyaretçiye olmalıdır.
  - Yeni bir eleman türü eklemek zordur: soyut ziyaretçiye yeni bir `visit` metodu, bütün somut ziyaretçilere de (gerekmese bile boş gövdeli) gerçekleme eklenir.
  - Etkileşim (ekrandan, 02:10:00): nesne yapısı kök düğüme `Accept(aVisitor)` gönderir; düğüm `VisitConcreteElementA(this)` çağırır; ziyaretçi düğüm üzerinde `OperationA()` yürütür; dönüşten sonra sıradaki düğüme geçilir. Hoca GoF şemasında eksik olan dönüş oklarını çizdi.
  - Kullanım: değişen arayüzlere sahip nesnelerden oluşan bir yapıda farklı işlemler gerekiyorsa VE yapıdaki türler değişmiyor ya da ender değişiyorsa.
- Visitor örneği: proje yönetimi yazılımı (ekrandan, 02:22:30)
  - «Abstract» `ProjectItem` (`name`, `description`); alt sınıfları `Project` (`projectItems`), `Task` (`projectItems`, `timeRequired`) ve `Deliverable` (`materialsCost`, `productionCost`). Proje görevleri, görevler ara çıktıları (deliverable) içerir.
  - Çözüm 1: `ProjectCalculator` süreyi ve maliyeti `instanceof` denetimleri ve tip dönüşümleriyle hesaplar (ekrandan, 02:30:00).
  - Çözüm 2: `ProjectItem`'a `accept(ProjectVisitor)` eklenir. «`interface`» `ProjectVisitor` (`visitDeliverable`, `visitTask`, `visitProject`) ve gerçeklemeleri `ProjectCostVisitor`, `ProjectTimeVisitor`. `Report` iki ziyaretçiyi kullanır.
  - `ProjectCostVisitor` kurucusunda `project.accept(this)` ile dolaşımı başlatır; `visitDeliverable` `cost += p.getMaterialsCost() + p.getProductionCost()` yapar; `visitTask` alt elemanlara `item.accept(this)` gönderir (ekrandan, 02:35:15).
  - `ProjectTimeVisitor`'da `visitDeliverable` boştur; `visitTask` `time += p.getTimeRequired()` sonrası alt elemanları dolaşır (ekrandan, 02:40:00).
  - İki çözüm aynı sonucu verdi; ikincisinde `instanceof` ve tip dönüşümü kalmadı, kod küçük ve yönetilebilir parçalara ayrıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- switch-case ve `instanceof` uyarı işaretidir
  - Kendinizi switch-case yazarken ya da `instanceof` ile tip denetleyip dönüştürürken buluyorsanız, büyük olasılıkla kalıtım ve çok biçimliliğe dayanan daha iyi bir tasarım gerekir.
- State'i Strategy'den ayıran şey durum geçiş kurallarıdır
  - Strategy dışarıdan verilir ve değişimi seyrektir; State'te context geçişleri kendisi yapar ve geçişler sıktır.
- Benzer şemalı kalıplar amaçlarıyla ayrılır
  - Builder ile Strategy'nin sınıf şemaları neredeyse aynıdır. Doğru kalıbı seçmek kalıpların ayrıntılarını ve çözdükleri problemleri iyi bilmeye bağlıdır; bunun kısa yolu yoktur, deneyimle kazanılır.
- Template Method'da şablon metot `final` olmalıdır
  - Şablon metot `final` yapılmazsa alt sınıflar onu ezip işi istedikleri gibi yürütebilir; bu kalıbın amacına aykırıdır.
- Ortak adım varsa Template Method, yoksa Strategy
  - Strategy'de işin tamamı farklı yapılır; Template Method'da bazı adımlar sabit sırada zorlanır, yalnız bazı alt adımlar değişir.
- Visitor'ın bedeli yeni eleman türü eklemenin zorluğudur
  - Eleman hiyerarşisi değişmeyecekse Visitor kullanılır; değişecekse kendinize fazladan iş çıkarırsınız.
- Sınıf şemalarında net gösterim
  - Soyut sınıf «`abstract`» stereotipiyle, soyut metot imzanın sonunda `{abstract}` ile yazılır; elle çizerken italik belli olmaz. Üye alanlar - (private), metotlar çoğunlukla + (public) gösterilir.

Kısa Tekrar Notları

- Strategy: aynı iş için birden fazla algoritma; context stratejiye bütün işi verir, strateji dışarıdan atanır ve çalışma anında değişebilir.
- Strategy zayıf yönleri: istemci stratejileri bilmek zorunda; push modelde haberleşme ek yükü.
- Strateji gerçeklemeleri Singleton olmaya adaydır (her ülkenin tek vergi kanunu).
- Her farklı mali işlem için ayrı Strategy kalıbı kurmak önerilir.
- State: durum değiştikçe davranış tamamen değişir; durumlar sınıf olur; geçiş kuralları context ya da durumlarda açıkça kodlanır.
- State ile Strategy: kim seçer (context kendisi / dışarıdan), geçiş sıklığı (sık / seyrek).
- PIM örneği: `CleanState.edit()` → `DIRTY`; `save()` → `CLEAN`. `DirtyState` boş bırakılırsa kirli durumdan temize geçiş kaybolur.
- Üst sınıf alt sınıfları bilmemelidir.
- Template Method: `final` şablon metot + `abstract` ilkel adımlar; sınıf `final` değildir.
- Veritabanı örneği: `doQuery()` sabit beş adım, `formatConnect()` ve `formatSelect()` VTYS'ye göre değişir.
- Visitor: elemanlarda `accept(visitor)`, ziyaretçide her eleman türü için `visitX(X)`; `accept` içinde `visitor.visitX(this)` (çift yönlendirme).
- Visitor ile yeni işlem eklemek kolay, yeni eleman türü eklemek zordur.
- Proje örneği: `ProjectCostVisitor` ve `ProjectTimeVisitor` `instanceof`'u ortadan kaldırır.

Detaylı Açıklamalar

- Strategy kalıbında context ile strateji arasındaki ilişki
  - GoF şemasında ok context'ten stratejiye doğrudur, ama strateji işini yaparken çoğu zaman context'ten bilgi almak zorundadır; bu yüzden ilişki pratikte genellikle çift yönlüdür. Farklı stratejiler context'ten farklı bilgilere ihtiyaç duyabilir. Bu yüzden context, stratejilerin isteyebileceği verilere erişim sağlayan getter metotları sunmalıdır. Hocaya göre bu metotlar context'i kullanan başka nesneler için de büyük olasılıkla zaten vardır; yoksa eklenir.
  - Hoca kalıbın yalnız tek bir strateji üyesiyle de, birden fazla stratejiyle de kurulabileceğini söyledi. Hangi stratejiye geçileceği, WordPad örneğinde olduğu gibi kullanıcı

girdisiyle dışarıdan belirlenir.

- Builder ile Strategy'nin ayrımı
  - Builder'da karmaşık bir ürün parça parça oluşturulur. Yönetici sınıf (`RTFReader`) metni okurken her bileşen için dönüştürücüye ayrı adım komutu verir ve sırayı kendisi denetler. Bazı metin biçimlerinde bazı bileşenlerin anlamı olmayabilir.
  - Strategy'de istemci "bu işi yap" deyip bütün işi strateji gerçeklemesine verir. Gerçekleme gerekli bilgiyi istemciden sorgulayarak alır ve işin denetim akışının çoğunu kendisi yürütür. İstemci işin arka planda nasıl yürüdüğünü bilmez.
- Finans örneğinde Strategy'nin kazancı
  - Switch-case çözümünde bir ülkeyle ilgili hata düzeltmek ya da gereksinim değişikliğini kodlamak için devasa bir metotta arama yapılır. Finans programında vergi dışında başka işlevler de olacağından `case TR` gibi parçalar programın her yerine dağılır.
  - Strategy çözümünde her bölgenin kodu kendi sınıfındadır. Geliştirici yalnız ilgili sınıfa odaklanır ve ülke çalışma anında `setTax()` ile değiştirilebilir.
- State kalıbında geçişleri kimin yaptığı
  - Slayta göre durum değişiminden genellikle context sorumludur. Hoca diğer seçeneği de açıkladı: durum sınıfları "benim için bu kadar, şu duruma geç" diyerek context'in durum atama metodunu çağırabilir. İki çalışma biçimi de mümkündür.
  - Bütün durumlarda ortak yapılan bir iş varsa soyut `State` sınıfında somut metot olarak kodlanabilir. Örneğin on durumdan sekizinde ortaksa üst sınıfa yazılır, farklı olan iki durumda ezilir (override). Ama durum geçişi üst sınıfa yazılmamalıdır, çünkü üst sınıf alt sınıflarını bilmemelidir.
- PIM örneğindeki düzeltme
  - Slayttaki ilk kodda `CleanState.save()` kayıttan sonra durumu `CLEAN` yapıyordu, `DirtyState` ise boş bir işaretleyici sınıftı. Hoca işaretleyici sınıfı Java'daki `Cloneable` ve `Serializable` gibi metotsuz işaretleyici arayüzlere benzetti. Ama bu durumda `DirtyState save()`'i soyut `State`'ten alır; o metotta durum geçişi olmadığı için kirli kayıt kaydedilse de temize dönmez.
  - Düzeltilmiş `DirtyState` (ekrandan, 01:10:00):

```
package dp.state.example;
import java.io.*;
public class DirtyState extends State {
    public void save(File f, Serializable s) throws IOException {
        super.save(f, s);
        StateFactory.setCurrentState(PIMApplication.CLEAN);
    }
}
```

- Kirli durumdayken `edit()` yine kirli durumda kalınmasını gerektirdiği için `DirtyState`'te `edit()` ezilmez. Hoca `StateFactory` adının şemada olmadığını, doğrusunun `PIMApplication.setCurrentState(...)` olduğunu söyleyip slaytları düzeltti. Durumlar Singleton yapılmak istenirse bir fabrika yararlı olabilir.

- Template Method'un kurallarla ilişkisi
  - Kalıp, üst sınıfın alt sınıfları bilmemesi kuralına uyar. Bununla birlikte alt sınıflara bazı işleri yapmayı yasaklar (şablonu değiştiremezler), bazı işleri de zorunlu kılar (soyut metotları gerçeklemek zorundadırlar).
  - `QueryTemplate`'te soyut metotlar paket görünürlüğündedir. Hoca bunun zorunlu olmadığını, kalıbın görünürlük konusunda bir şey dayatmadığını söyledi.
- Visitor'da çift yönlendirme
  - Her düğüm ziyaretçiyi kabul eder ama "ben bir atama düğümüyüm, beni atama düğümü olarak ziyaret et" diyerek ziyaretçinin kendi türüne özel metodunu `this` ile çağırır. Böylece her parçada yalnız gerekli işlem yapılır. Bir düğüm türüyle ilgili bütün işlemler somut ziyaretçi sınıflarında toplanır.
  - Proje örneğinde ilk çözümdeki `if (item instanceof Task) time += calculateTimeRequired((Task) item);` gibi satırlar (ekrandan, 02:30:00) ikinci çözümde gereksiz olur, çünkü metot imzası hangi düğüm türünün dolaşıldığını zaten söyler.
  - Raporda iki çözüm aynı süre ve maliyet sonucunu verdi.
- Kalıpları öğrenmenin yararı (öğrenci sorusu üzerine)

  - Kalıpları bilmek, bir problemle karşılaşınca elde hazır bir çözüm repertuarı olması demektir; hoca bunu yaz, kış ve yüksek performans lastiği arasından uygun olanı seçmeye benzetti. Başlangıçta kalıpların hangi sorunları çözdüğünü ilk belirtileri düzeyinde bilmek yeterlidir; ayrıntı kullandıkça gelir.
  - Kalıplar değişen gereksinimlere karşı esneklik sağlar; bu yüzden kişisel projelerde de yararlıdır. Açık kaynak projelerde "bu kısım Strategy ile yapılmıştır" gibi ifadelerle karşılaşılır ve kalıbın bilinmesi beklenir.
  - Tasarım kalıpları dilden bağımsızdır. Dile bağlı küçük farklar vardır (örneğin Memento'nun en sağlam nasıl kodlanacağı), ama kalıplar herhangi bir dille öğrenilebilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.